

Guía Avanzada de Recorridos de Matrices en C

Estrategias algorítmicas, aplicaciones prácticas y fundamentación matemática para docentes.

1. Recorrido Total (Por Filas)

Técnica: Visita cada elemento secuencialmente de izquierda a derecha, moviéndose de arriba a abajo.

Fórmula / Modelo: Índices ordenados lexicográficamente: (i, j) donde $i \in [0, M-1]$ y $j \in [0, N-1]$.

Uso común: Inicialización de datos, cálculo de promedios, impresión de tablas.

```
for (int i = 0; i < FILAS; i++) {
    for (int j = 0; j < COLUMNAS; j++) {
        printf("%d ", matriz[i][j]);
    }
}
```

2. Recorrido Inverso

Técnica: Visita la matriz empezando desde la última celda $[FILAS-1][COLUMNAS-1]$ hacia atrás hasta llegar a $[0][0]$.

Fórmula / Modelo: Índices inversos decrecientes desde $(M-1, N-1)$ hasta $(0,0)$.

Uso común: Inversión de imágenes, algoritmos de programación dinámica en reversa.

```
for (int i = FILAS - 1; i >= 0; i--) {
    for (int j = COLUMNAS - 1; j >= 0; j--) {
        printf("%d ", matriz[i][j]);
    }
}
```

3. Diagonal Principal

Técnica: Cruza la matriz cuadrada desde la esquina superior izquierda a la inferior derecha.

Fórmula / Modelo: Definida matemáticamente por el conjunto de celdas donde el índice de fila coincide con la columna: $i == j$.

Uso común: Cálculo de la traza de una matriz en álgebra lineal, filtros gráficos diagonales.

```
for (int i = 0; i < N; i++) {
    printf("%d ", matriz[i][i]);
}
```

4. Diagonal Secundaria

Técnica: Cruza la matriz desde la esquina superior derecha hasta la inferior izquierda.

Fórmula / Modelo: Definida por la restricción lineal donde la suma de los índices es constante: $i + j == N - 1$, despejando $j = N - 1 - i$.

Uso común: Álgebra lineal, transformaciones espaciales simétricas.

```
for (int i = 0; i < N; i++) {
    printf("%d ", matriz[i][N - 1 - i]);
}
```

5. Mitad Diagonal (Triangular Superior)

Técnica: Recorre los elementos situados en la diagonal principal y por encima de ella.

Fórmula / Modelo: El espacio de búsqueda está acotado por la desigualdad geométrica: $j \geq i$.

Uso común: Resolución de sistemas de ecuaciones lineales por eliminación Gaussiana, optimización de almacenamiento en matrices simétricas.

```
for (int i = 0; i < N; i++) {
    for (int j = i; j < N; j++) {
        printf("%d ", matriz[i][j]);
    }
}
```

6. Mitad Diagonal (Triangular Inferior)

Técnica: Recorre los elementos que están en la diagonal principal y por debajo de ella.

Fórmula / Modelo: El espacio de búsqueda está acotado por la desigualdad geométrica: $j \leq i$.

Uso común: Factorización LU de matrices, optimización de cálculos repetitivos.

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j <= i; j++) {
        printf("%d ", matriz[i][j]);
    }
}
```

7. Desplazamiento Adyacente (Análisis de Vecindad de 8 Conexiones)

Técnica: Inspecciona el entorno local de una coordenada central fija $[r][c]$, evaluando de forma controlada sus celdas vecinas.

Fórmula / Modelo: Modelo Matemático de Desplazamiento Relativo:

Sea la celda origen $V_0 = (r, c)$. El conjunto de celdas vecinas inmediatas se define como:

$$N_8(r, c) = \{ (r + \Delta f, c + \Delta c) \mid \Delta f, \Delta c \in \{-1, 0, 1\} \wedge (\Delta f, \Delta c) \neq (0, 0) \}$$

Para evitar desbordamientos de memoria en los bordes de la matriz (IndexOutOfBounds), cada vecino candidato (v_f, v_c) debe pasar a través del siguiente **Filtro de Acotamiento Lógico (Espacio M x N)**:

$$F(v_f, v_c) = (0 \leq v_f < M) \wedge (0 \leq v_c < N)$$

Uso común: Algoritmo del Buscaminas (contar minas), Autómatas Celulares (Juego de la Vida de Conway), Procesamiento de Imágenes (Filtros de Convolución, Sobel, desenfoque).

```
// r y c representan la celda central del análisis
int df[] = {-1, -1, -1, 0, 0, 1, 1, 1}; // Desplazamientos en filas
int dc[] = {-1, 0, 1, -1, 1, -1, 0, 1}; // Desplazamientos en columnas

for (int v = 0; v < 8; v++) {
    int nueva_f = r + df[v];
    int nueva_c = c + dc[v];

    // Filtro matemático de fronteras
    if (nueva_f >= 0 && nueva_f < FILAS && nueva_c >= 0 && nueva_c < COLUMNAS) {
        printf("Vecino válido en: [%d][%d] = %d\n", nueva_f, nueva_c, matriz[nueva_f][nueva_c]);
    }
}
```

8. Recorrido Circular o en Espiral

Técnica: Lee las capas perimetrales de la matriz desde el exterior reduciendo las dimensiones hacia el centro dinámicamente.

Fórmula / Modelo: Uso de 4 límites móviles variables (top, bottom, left, right) que se contraen secuencialmente de manera lineal.

Uso común: Cifrado de datos en matrices (Transposición), organización espaciotécnica de vectores.

```
int top = 0, bottom = FILAS - 1;
int left = 0, right = COLUMNAS - 1;
while (top <= bottom && left <= right) {
    for (int i = left; i <= right; i++) printf("%d ", mat[top][i]);
    top++;
    for (int i = top; i <= bottom; i++) printf("%d ", mat[i][right]);
    right--;
    if (top <= bottom) {
        for (int i = right; i >= left; i--) printf("%d ", mat[bottom][i]);
        bottom--;
    }
    if (left <= right) {
        for (int i = bottom; i >= top; i--) printf("%d ", mat[i][left]);
        left++;
    }
}
```